

IDA-VIRUMAA KUTSEHARIDUSKESKUS

Артём Патока, Арсений Сергеев

Группа: **SPTV22**

ИНФОРМАЦИОННАЯ СИСТЕМА И ВЕБ-ПЛАТФОРМА ЭЛЕКТРОННОЙ ТОРГОВЛИ СПОРТИВНОГО ПИТАНИЯ „SPSHOP"

Выпускная / Курсовая работа

Ссылка на Figma: [Figma

Design](<https://www.figma.com/design/d2zZs9u3NjY0mb1wiZmDA3/Untitled?m=auto&t=KRpxN2Bh6FRghPfH-6>)

Руководитель: Сергей Лобышев

Нарва 2026

...

СОДЕРЖАНИЕ

[ВВЕДЕНИЕ](#введение)

[1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ ПРОЕКТА](#1-техническое-задание-проекта)

1.1. Постановка задачи

 [1.2. SWOT-анализ](#12-swot-анализ)

[2. ПЛАНИРОВАНИЕ ПРОЕКТА](#2-планирование-проекта)

2.1. Члены команды и распределение задач

 [2.2. Требования к системе](#22-требования-к-системе)

 [2.3. Пользовательские сценарии](#23-пользовательские-сценарии)

 [2.4. Дизайн и стилистика проекта](#24-дизайн-и-стилистика-проекта)

[2.5. Бюджет и планирование времени](#25-бюджет-и-планирование-времени)

 [2.6. Управление рисками](#26-управление-рисками)

[3. РЕАЛИЗАЦИЯ ПРОЕКТА](#3-реализация-проекта)

 [3.1. Этапы реализации проекта](#31-этапы-реализации-проекта)

 [3.2. Инструменты разработки](#32-инструменты-разработки)

3.3. Описание архитектуры и программных модулей

3.3.1. Разработка Backend](#331-разработка-backend)

3.3.2. База данных](#332-база-данных)

3.3.3. Права доступа и безопасность (Security & RBAC)(#333-права-доступа-и-безопасность-security--rbac)

 [3.3.4. Контроллеры и API](#334-контроллеры-и-api)

3.3.5. Реализация авторизации, регистрации и сессий](#335-реализация-авторизации-регистрации-и-сессий)

 [3.3.6. Разработка Frontend](#336-разработка-frontend)

3.3.7. Интеграция с платежным шлюзом Stripe](#337-интеграция-с-платежным-шлюзом-stripe)

3.3.8. Корзина товаров и управление заказами](#338-корзина-товаров-и-управление-заказами)

3.3.9. Интеграция с почтовыми отправлениями (OmniVa и SmartPost)](#339-интеграция-с-почтовыми-отправлениями-omniva-и-smartpost)

3.3.10. Панель администратора](#3310-панель-администратора)

3.3.11. Развертывание на хостинге](#3311-развертывание-на-хостинге)

Проект **SPSHOP** представляет собой полнофункциональную веб-платформу электронной коммерции, разработанную с использованием современного стека технологий на базе платформы **Java / Spring Boot 3.x**. Решение ориентировано на розничную дистрибуцию спортивного питания, включая такие категории товаров, как протеины, креатин, предтренировочные комплексы, аминокислоты и жиросжигатели. Платформа

предназначена как для конечных покупателей, так и для менеджеров и администраторов магазина.

Основная цель проекта — создание высокопроизводительного интернет-магазина премиум-класса с интеграцией локальных почтовых сетей Эстонии (Omniva, SmartPost), международных платежных шлюзов (Stripe) и продвинутой системы администрирования. Важнейшим приоритетом разработки являлось обеспечение высочайшей стабильности и безопасности ПО, что подтверждается наличием комплексной тестовой базы из **428 тестов** (модульные, интеграционные и сквозные E2E сценарии в браузере).

В рамках проекта была реализована полная цепочка электронной торговли: от регистрации нового покупателя и просмотра каталога товаров до оформления заказа, выбора почтового автомата для получения посылки и совершения безопасного онлайн-платежа через систему Stripe. Разделённый интерфейс (пользовательская и административная части) обеспечивает четкое разграничение полномочий и предоставляет администраторам инструменты для управления ассортиментом, заказами и пользователями.

Дизайн сайта выполнен с использованием современной темной эстетики (глассморфизм), создающей премиальный визуальный облик и стимулирующей взаимодействие с платформой. Для реализации были применены технологии Thymeleaf-шаблонизации, чистый CSS с анимациями, а также JavaScript-компоненты для асинхронного взаимодействия с сервером.

Данная работа описывает полный жизненный цикл разработки веб-приложения: от первоначального планирования и постановки задачи до реализации каждого программного модуля и итогового тестирования.

1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ ПРОЕКТА

1.1. Постановка задачи

Целью проекта является проектирование и разработка информационной системы электронной коммерции «SPSHOP». Веб-приложение должно решать следующие задачи:

1. Для неавторизованных пользователей (гостей):

- Свободный доступ к просмотру каталога товаров спортивного питания без необходимости регистрации.
- Фильтрация товаров по категориям (протеины, ВСАА, жиросжигатели и др.) и динамический поиск товаров по названию.
- Прохождение регистрации (форма с валидацией полей: имя, email, телефон, пароль) и авторизации.
- Возможность сброса пароля через email-ссылку с токеном.

2. Для зарегистрированных покупателей:

- Управление корзиной товаров: добавление, редактирование количества единиц товара, удаление позиций.
- Динамический пересчёт суммы корзины в реальном времени без перезагрузки страницы (AJAX).
- Выбор способов доставки из встроенных справочников почтовых автоматов (Omniva, SmartPost).
- Оформление заказов и безопасная онлайн-оплата через платежный шлюз Stripe с использованием Stripe Elements.
- Просмотр личной истории заказов и отслеживание статуса доставки.
- Управление профилем пользователя.

3. Для администраторов системы:

- Управление категориями товаров: создание, активация/деактивация, загрузка изображений.
- Управление товарным ассортиментом: добавление новых позиций, редактирование цены, стока, скидок, описаний, изображений.
- Мониторинг всех заказов в системе и обновление статуса выполнения (Ordered, Payment Approved, Shipped, Delivered, Cancelled).
- Управление аккаунтами пользователей: просмотр списка, блокировка и разблокировка при превышении числа попыток ввода пароля.
- Аналитический дашборд с показателями продаж, количества пользователей и активных заказов.

Требования к технической среде определяют, что приложение должно быть развернуто на облачной платформе (Microsoft Azure), использовать управляемую СУБД MySQL, поддерживать работу под нагрузкой и обеспечивать защищенное соединение HTTPS.

1.2. SWOT-анализ

Для оценки рыночных перспектив и надежности платформы SPSHOP был проведен SWOT-анализ, результаты которого представлены в Таблице 1.

Таблица 1. SWOT-анализ проекта

Сильные стороны	Слабые стороны
1. Премиальный визуальный дизайн на основе глассморфизма и темной темы. 2. Безопасность транзакций: шифрование ID платежей через `SecurityUtils`, интеграция с официальным Stripe API. 3. Полное тестовое покрытие (428 тестов) снижает вероятность дефектов до минимума. 4. Локализация методов доставки (Omniva/SmartPost) под рынок Эстонии. 5. Развернутое облачное решение на Azure с SSL-сертификатом. 6. Автоматическая блокировка учетных записей при превышении числа попыток ввода пароля.	1. Высокая зависимость от стабильности внешних API (Stripe, API почтовых служб). 2. Повышенные требования к вычислительным ресурсам сервера (JVM) по сравнению с легковесными JS-решениями. 3. Поддержка только одного языка интерфейса.
Возможности	Угрозы
1. Масштабирование на международный рынок за счет внедрения мультиязычности (i18n). 2. Добавление программы лояльности (кешбэк, бонусные баллы). 3. Интеграция интерактивной карты посылочных автоматов для наглядного выбора доставки. 4. Подключение мобильных платежей (Apple Pay, Google Pay) через Stripe.	1. Высокая конкуренция со стороны глобальных e-commerce платформ (Amazon, eBay). 2. Изменение требований безопасности платежных систем (PCI DSS), требующее регулярных обновлений ПО. 3. Риски кибератак (брутфорс, XSS, SQL-инъекции).

2. ПЛАНИРОВАНИЕ ПРОЕКТА

2.1. Члены команды и распределение задач

Разработка системы велась группой из двух специалистов. Каждый участник имел зоны ответственности, при этом ключевые модули разрабатывались совместно. Распределение ролей и ответственности представлено в Таблице 2.

Таблица 2. Распределение задач в команде

Задача	Артём Патока	Арсений Сергеев
Разработка структуры базы данных (MySQL, JPA)	+	
Настройка Spring Security & RBAC	+	
Реализация REST API и бизнес-логики (Service layer)	+	
Верстка Thymeleaf шаблонов и стилизация UI (CSS)		+
Интеграция Stripe SDK и AJAX корзины	+	+
Разработка макета интерфейса (Figma)		+
Написание Unit и Integration тестов (JUnit/Mockito)		+
Автоматизация E2E тестов в браузере (Playwright)	+	
Тестирование почтового сервиса и логистического выбора		+

Развертывание системы на Azure и настройка Docker	+	
Управление задачами в Jira и ведение документации		+

Планирование задач велось с использованием **Jira** (Atlassian) в стиле Kanban-доски. Каждая задача включала описание, приоритет, оценку времени и ответственного исполнителя. Дизайн интерфейса создавался в **Figma** с предварительным согласованием макетов страниц с командой.

2.2. Требования к системе

Функциональные требования:

2.2.1. Система должна поддерживать авторизацию на базе сессий HTTP и Spring Security (form-based login).

2.2.2. Пароли пользователей должны быть захешированы с использованием алгоритма BCrypt с солью (силой не менее 10 раундов).

2.2.3. Система должна автоматически блокировать учетную запись пользователя при превышении 3 неверных попыток ввода пароля, фиксируя время блокировки. Разблокировка происходит автоматически через 24 часа или вручную администратором.

2.2.4. Корзина должна динамически пересчитывать общую стоимость товаров без перезагрузки страницы (AJAX + Fetch API).

2.2.5. Платежный модуль должен работать через официальный Stripe Java SDK. Платежные данные карт клиента не должны проходить через собственные серверы приложения — используется Stripe Elements (iframe).

2.2.6. Транзакционный идентификатор Stripe должен сохраняться в базе данных в зашифрованном виде.

2.2.7. Форма оформления заказа должна предоставлять выбор между почтовыми службами Omniva и SmartPost с указанием конкретного терминала.

Нефункциональные требования:

2.2.8. Совместимость со всеми актуальными браузерами (Chrome, Firefox, Edge, Safari).

2.2.9. Время отклика сервера на стандартные запросы не должно превышать 200 мс при нормальной нагрузке.

2.2.10. Полная адаптивность интерфейса под мобильные устройства (смартфоны, планшеты) через CSS медиа-запросы.

2.2.11. Веб-приложение должно работать исключительно по протоколу HTTPS.

2.2.12. Конфиденциальные параметры (ключи Stripe, реквизиты БД) должны передаваться только через переменные окружения, а не хардкодироваться в исходном коде.

2.3. Пользовательские сценарии

Для описания взаимодействия пользователей с платформой были выделены основные роли: Гость, Клиент и Администратор (см. Таблицы 3, 4, 5).

Таблица 3. Сценарии неавторизованного пользователя (Гость)

№	Действие	Описание
1	Просмотр каталога	Гость открывает главную страницу, пролистывает список спортивного питания по всем категориям.
2	Фильтрация по категории	Гость нажимает на категорию (например, «Протеин») — система отображает только товары этой категории.
3	Поиск товаров	Гость вводит запрос в строку поиска в шапке сайта и мгновенно получает отфильтрованный список.

4	Просмотр карточки товара	Гость открывает детальную страницу продукта с описанием, ценой, скидкой и фотографией.
5	Регистрация	Гость нажимает кнопку «Register», заполняет форму (Имя, Email, Телефон, Пароль) и создает аккаунт.
6	Авторизация	Гость вводит логин/пароль. При ошибке получает сообщение. После 3 попыток аккаунт блокируется.
7	Сброс пароля	Гость вводит email и получает письмо со ссылкой сброса пароля с уникальным токеном.

Таблица 4. Сценарии зарегистрированного пользователя (Клиент)

№	Действие	Описание
1	Добавление в корзину	Клиент нажимает «Add to Cart» на карточке товара. Счетчик корзины в шапке обновляется мгновенно.
2	Управление корзиной	Клиент переходит в корзину, увеличивает или уменьшает количество единиц каждого товара. Итоговая сумма пересчитывается.

3	Оформление заказа	Клиент нажимает «Checkout», заполняет форму адреса, выбирает провайдера (Omniva/SmartPost) и терминал.
4	Оплата через Stripe	Клиент вводит данные карты в Stripe Elements форму. Сервер создает PaymentIntent и возвращает clientSecret.
5	Подтверждение заказа	После успешной оплаты Stripe вызывает подтверждение, корзина очищается, заказ сохраняется в БД.
6	История заказов	Клиент просматривает список своих заказов со статусами: Ordered, Payment Approved, Shipped, Delivered.

Таблица 5. Сценарии администратора

№	Действие	Описание
1	Управление каталогом	Администратор добавляет новые продукты с описанием, ценой, скидкой, стоком и изображением.
2	Управление категориями	Создание, активация и деактивация категорий товаров.

3	Управление заказами	Просмотр всех заказов в системе, обновление статуса доставки.
4	Управление пользователями	Просмотр всех покупателей, разблокировка заблокированных аккаунтов.
5	Аналитика	Просмотр дашборда с общим числом заказов, пользователей, категорий и товаров.

2.4. Дизайн и стилистика проекта

Для сайта выбран уникальный дизайн в стиле **Glassmorphism Dark UI**. Данное направление в веб-дизайне отличается использованием полупрозрачных элементов с размытым задним фоном, что создаёт ощущение многослойности интерфейса и визуальной глубины.

Логотип проекта SPSHOP отражает современную спортивную эстетику:



Рисунок. Логотип проекта SPSHOP

Цветовая палитра основана на гармоничных HSL-цветах с использованием размытия заднего фона (`'backdrop-filter: blur(12px)'`), создающего ощущение глубины и премиальности. Применение технологии CSS `'backdrop-filter'` обеспечивает эффект «матового стекла» поверх темного фоновой слоя.

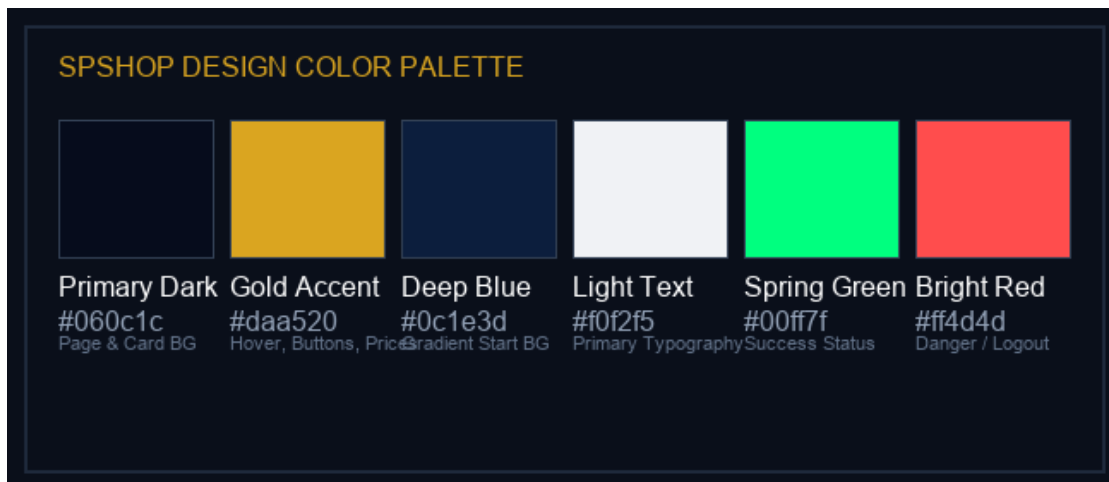


Рисунок. Цветовая палитра дизайна

Основные шрифты: **Inter** (для основного текстового наполнения) и **Outfit** (для заголовков и акцентных элементов) — оба являются Google Fonts с широкой поддержкой символов.

Компоненты оформления:

- Основной фон: глубокий темный цвет (`#0b0f19`), имитирующий ночное пространство.
- Карточки товаров: полупрозрачные контейнеры со светлыми границами (`rgba(255, 255, 255, 0.05)`) и мягкими тенями.
- Акцентные элементы (кнопки «Добавить в корзину», индикаторы статуса): насыщенный синий/голубой градиент (`hsl(210, 90%, 55%)` → `hsl(240, 80%, 65%)`).
- Анимации: плавные CSS-переходы `transition: 0.2s ease` на всех интерактивных элементах.
- Формы: поля ввода с темным фоном и подсветкой активного поля через `box-shadow` в акцентном цвете.

2.4.1. Макет интерфейса в Figma

Перед началом верстки веб-страниц был разработан интерактивный прототип интерфейса в Figma, доступный по ссылке: [Figma

Design](<https://www.figma.com/design/d2zZs9u3NjY0mb1wiZmDA3/Untitled?m=auto&t=KRpxN2Bh6FRghPfH-6>). Макет включает в себя структуру главного каталога товаров, карточек продуктов, корзины и панели администратора.

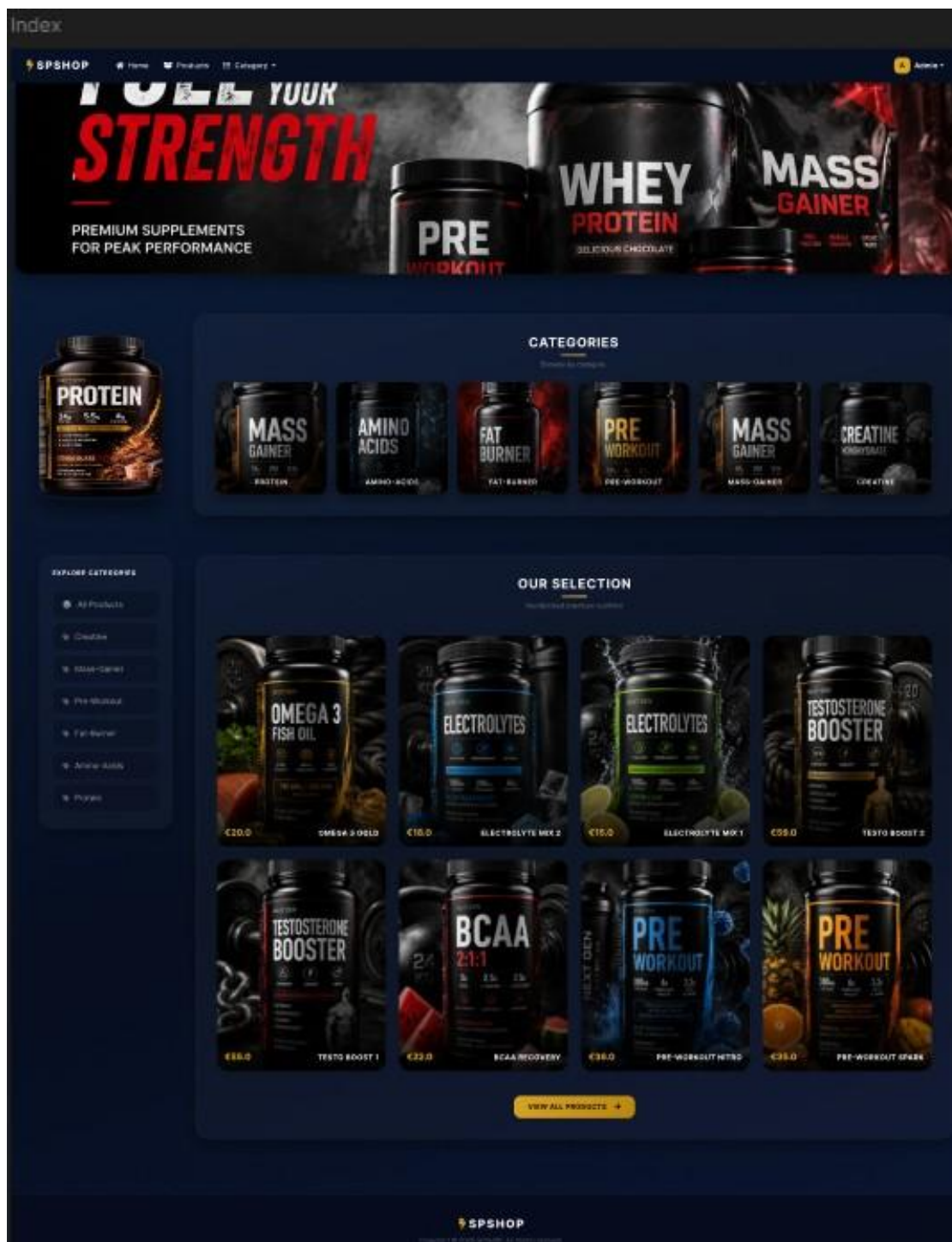


Рисунок. Макет главной страницы интернет-магазина в Figma

2.5. Бюджет и планирование времени

Расчет бюджета и затрат времени производился из расчета ставки 9.00 евро в час (Таблицы 6, 7, 8).

Таблица 6. Планирование и затраты времени (в часах)

№	Задача	Запланировано часов	Потрачено часов
1	Проектирование базы данных и архитектуры	80	78
2	Разработка Backend (Spring Boot, Security)	210	205
3	Разработка Frontend шаблонов (Thymeleaf, CSS)	180	182
4	Интеграция Stripe и Логистики	70	68
5	Написание автоматических тестов	110	115
6	Документирование и развертывание (Azure)	40	38
Итого	Всего:	690	686

Таблица 7. Бюджет на реализацию задач

№	Задача	Потрачено часов	Цена за час	Общая стоимость
1	Проектирование	78	9.00 €	702.00 €
2	Разработка Backend	205	9.00 €	1845.00 €

3	Разработка Frontend	182	9.00 €	1638.00 €
4	Интеграция сервисов	68	9.00 €	612.00 €
5	Тестирование	115	9.00 €	1035.00 €
6	Документирование	38	9.00 €	342.00 €
Итог	Всего:	686		6174.00 €

Таблица 8. Выработка участников и оплата

№	Участник команды	Отработано часов	Сумма оплаты
1	Артём Патока	345	3105.00 €
2	Арсений Сергеев	341	3069.00 €
Итог	Всего:	686	6174.00 €

2.6. Управление рисками

При реализации проектов электронной торговли безопасность персональных данных и платежей имеет первостепенное значение. В Таблице 9 приведены возможные технические и операционные риски и принятые меры по их снижению.

Таблица 9. Управление техническими рисками

Название риска	Вероятность	Ущерб	Меры по предотвращению
Утечка паролей пользователей	Низкая	Высокий	Использование алгоритма BCrypt с динамической солью. Пароли не

			хранятся в открытом виде.
SQL-инъекции	Низкая	Высокий	Spring Data JPA (Hibernate) с параметризованными JPQL-запросами вместо сырого SQL.
Мошенничество с оплатой	Средняя	Высокий	Платежи обрабатываются через Stripe Elements (iframe). Данные карт не проходят через серверы SPSHOP. В БД сохраняется только зашифрованный ID транзакции.
Брутфорс авторизации	Средняя	Высокий	Автоматическая блокировка аккаунта после 3 неудачных попыток входа на 24 часа.
Перегрузка БД при росте трафика	Низкая	Средний	Пул соединений HikariCP, оптимизация индексов MySQL, кеширование сессий.
Недоступность внешних API (Stripe, Omniva)	Средняя	Средний	Отображение информативного сообщения об ошибке пользователю. Повторные запросы при тайм-аутах.
Ошибки	Низкая	Средний	Использование

развёртывания на Azure			Docker-контейнеризации для воспроизводимости среды. Резервное копирование БД.
------------------------	--	--	---

3. РЕАЛИЗАЦИЯ ПРОЕКТА

3.1. Этапы реализации проекта

Разработка SPSHOP велась итеративно, с последовательным наращиванием функциональности. Были выделены следующие этапы:

3.1.1. Инициализация проекта: На данном этапе была создана базовая структура Maven-проекта с использованием веб-конфигуратора Spring Initializr. В качестве основы была выбрана платформа Java 17 LTS и Spring Boot 3.3.2. Были подключены ключевые зависимости (стартеры) для обеспечения работы различных систем проекта: `spring-boot-starter-web` для обработки HTTP-запросов и реализации REST API, `spring-boot-starter-security` для обеспечения авторизации и защиты данных, `spring-boot-starter-data-jpa` для взаимодействия с СУБД, `spring-boot-starter-thymeleaf` в качестве основного шаблонизатора веб-страниц, а также `spring-boot-starter-mail` для отправки электронных писем при сбросе паролей. Также была настроена структура каталогов бэкенд-приложения в соответствии с архитектурой MVC.

3.1.2. Проектирование базы данных: На этом этапе была разработана логическая и физическая схема базы данных. Были спроектированы основные сущности (JPA Entities), такие как `User`, `Product`, `Category`, `Cart`, `ProductOrder` и `OrderAddress`. Настройка связей проводилась с использованием аннотаций JPA (`@OneToMany`, `@ManyToOne`, `@OneToOne`). Особое внимание было уделено каскадному обновлению и удалению записей (`CascadeType.ALL`, `orphanRemoval=true`) для обеспечения целостности данных при удалении пользователей или очистке корзин. Автоматическая генерация и обновление таблиц были настроены через Hibernate DDL.

3.1.3. Разработка сервисного слоя (Service Layer): На третьем этапе была реализована основная бизнес-логика приложения. Были созданы интерфейсы и их реализации для управления пользователями (регистрация, блокировка, сброс пароля), каталогом товаров (добавление, редактирование, фильтрация, расчет скидок), корзиной покупок (добавление позиций, динамическое изменение количества) и оформлением заказов. Также была реализована интеграция с API платежного шлюза Stripe и алгоритмы работы со справочниками логистических служб доставки.

3.1.4. Настройка системы авторизации и безопасности: На данном шаге была выполнена конфигурация Spring Security. Были настроены правила фильтрации URL-адресов на основе ролей пользователей (клиент и администратор), отключен несанкционированный доступ к административным панелям и страницам оформления заказа. Была реализована форма входа (form-based login) с пользовательскими обработчиками успеха и ошибок аутентификации. Для защиты от брутфорса был внедрен сервис подсчета неудачных попыток входа с автоматической блокировкой аккаунтов. Все пароли были защищены шифрованием BCrypt.

3.1.5. Разработка пользовательского интерфейса (Frontend): На пятом этапе была выполнена верстка веб-страниц с использованием Thymeleaf и чистого CSS. Были реализованы шаблоны для главного каталога, карточки товара, корзины, форм авторизации и регистрации, а также личного кабинета пользователя. Особое внимание было уделено дизайну в стиле Glassmorphism Dark UI, адаптивной верстке для мобильных устройств и интеграции JavaScript (Fetch API / AJAX) для динамического обновления элементов корзины без перезагрузки страниц.

3.1.6. Интеграция платежного шлюза Stripe: На шестом этапе была реализована интеграция со Stripe Java SDK. Был разработан механизм создания сессий оплаты и объектов `PaymentIntent` на стороне сервера, а на фронтенде интегрирована безопасная форма Stripe Elements. Для защиты финансовых транзакций была внедрена утилита шифрования Stripe Transaction ID перед сохранением в БД с помощью алгоритма AES/CBC/PKCS5Padding.

3.1.7. Интеграция логистических служб: Была разработана система доставки посылок через локальных операторов Эстонии — Omniva и SmartPost. С помощью парсинга

внешних справочников или статических реестров была реализована динамическая загрузка списков почтовых терминалов по городам. Пользователю в процессе оформления заказа предоставляется удобный интерфейс для выбора провайдера и конкретного терминала, адрес которого связывается с заказом через сущность `OrderAddress`.

3.1.8. Тестирование и верификация: На этапе тестирования было разработано 428 тестовых сценариев. Написаны модульные тесты для репозиторий и сервисов с использованием JUnit 5 и Mockito для изоляции зависимостей. Для сквозного (E2E) тестирования была интегрирована библиотека Microsoft Playwright Java, позволяющая запускать браузер Chromium в фоновом режиме и проходить полные сценарии от регистрации до оплаты заказа с сохранением видеозаписей тестов.

3.1.9. Развертывание в облачной среде: Финальный этап включал подготовку приложения к промышленной эксплуатации. Был разработан `Dockerfile` для сборки контейнера с веб-приложением. База данных была перенесена на управляемый облачный сервис Azure Database for MySQL, а само приложение было развернуто в Microsoft Azure App Service. Был подключен SSL-сертификат для обеспечения HTTPS-соединения и настроены переменные окружения для защиты секретных ключей Stripe и реквизитов подключения к БД.

3.2. Инструменты разработки

Таблица 10. Технологический стек проекта

Категория	Технология / Инструмент	Версия	Назначение
Язык программирования	Java	JDK 17	Основной язык бэкенда
Фреймворк	Spring Boot	3.3.2	Ядро приложения, конфигурация
Безопасность	Spring Security	6.x	RBAC, авторизация, сессии

ORM	Spring Data JPA / Hibernate	6.x	Работа с базой данных
СУБД (prod)	MySQL Server	8.0	Основное хранилище данных
СУБД (tests)	H2 Database	2.x	In-memory БД для тестов
Шаблонизатор	Thymeleaf	3.x	Генерация HTML на стороне сервера
Система сборки	Apache Maven	3.9.x	Управление зависимостями, сборка
Платежный шлюз	Stripe Java SDK	24.0.0	Обработка онлайн- платежей
Тестирование (unit)	JUnit 5 + Mockito	5.x	Модульные тесты
Тестирование (E2E)	Microsoft Playwright Java	1.x	Браузерные тесты
Контейнеризация	Docker	latest	Деплой приложения
Облако	Microsoft Azure	—	Хостинг и управляемая БД
Дизайн	Figma	—	Разработка макетов UI
Управление задачами	Jira (Atlassian)	—	Kanban-доска, трекинг задач
IDE	IntelliJ IDEA	2024.x	Среда разработки

3.2.1. Использование системы контроля версий Git и репозитория GitHub

В процессе совместной разработки проекта SPSHOP активно использовалась распределённая система контроля версий **Git** совместно с хостингом репозитория **GitHub**. Это позволило организовать совместную работу команды над кодом, предотвратить конфликты при слиянии изменений и обеспечить прозрачную историю разработки.

- Ветвление (Branching): Для разработки каждой новой функциональности (например, интеграция Stripe, настройка логистики, написание E2E тестов) создавались отдельные ветки от основной ветви `main`.
- Pull Requests: Интеграция изменений в ветвь `main` выполнялась исключительно через пул-реквесты после обязательного прохождения локального тестирования и код-ревью другими членами команды.
- GitHub Issues: Для отслеживания задач, багов и планирования спринтов использовалась интеграция GitHub с Kanban-доской проекта.

3.2.2. Применение ChatGPT Plus для генерации графических ассетов

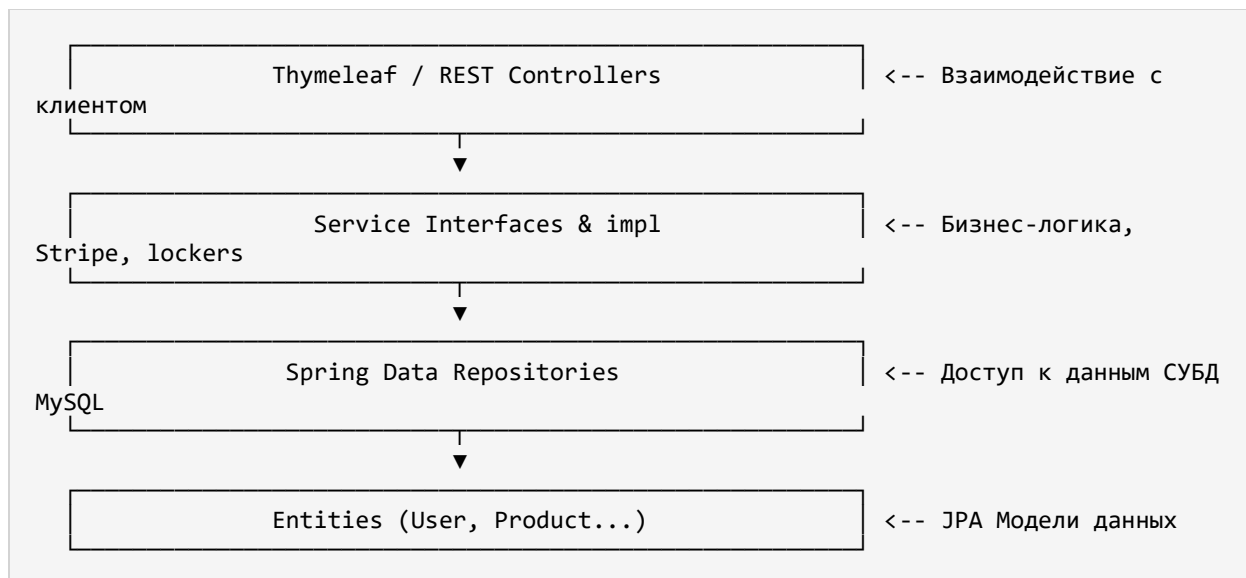
Для создания визуального контента проекта использовалась нейросетевая модель **ChatGPT Plus** (а именно инструмент **DALL-E 3**). С её помощью были сгенерированы уникальные графические элементы:

- Логотип проекта (`logo.png`): Был сгенерирован минималистичный логотип спортивной тематики в темных тонах, соответствующий выбранной теме Glassmorphism.
- Баннеры и фоны: Изображения для слайдеров и промо-акций товаров на главной странице каталога были созданы с учётом общей цветовой гаммы интернет-магазина.
- Применение генеративного ИИ позволило сократить расходы на услуги профессиональных дизайнеров и ускорить подготовку проекта к запуску.

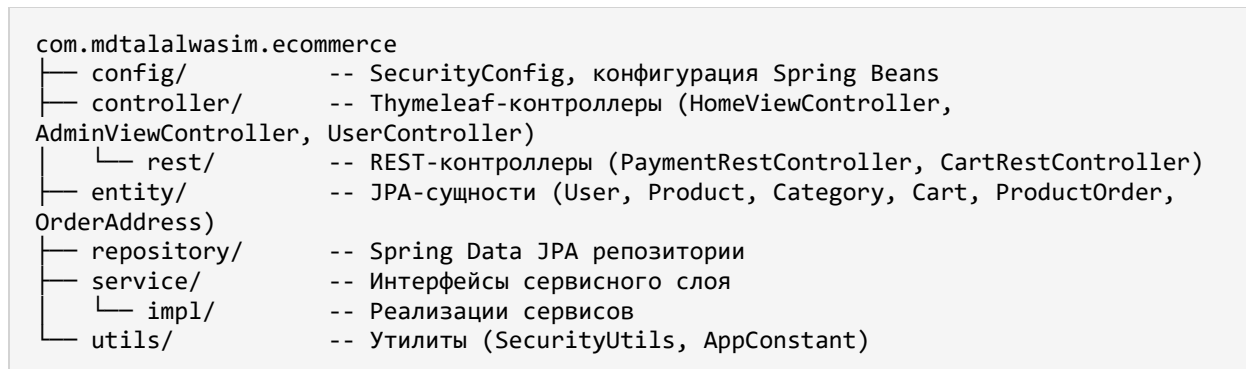
3.3. Описание архитектуры и программных модулей

3.3.1. Разработка Backend

Бэкенд приложения построен по классической многослойной архитектуре **MVC (Model-View-Controller)** с четким разделением ответственности между слоями. Такой подход обеспечивает высокую сопровождаемость, тестируемость и возможность независимого развития каждого слоя.



Структура пакетов Java-приложения организована следующим образом:



Центральная точка входа — класс `EcommerceStoreApplication.java`, аннотированный `@SpringBootApplication`. Он запускает контекст Spring и встроенный сервер Tomcat.

3.3.2. База данных

Схема базы данных MySQL состоит из **6 основных таблиц**, спроектированных с помощью Hibernate/JPA и аннотации `@Entity`. Генерация DDL-скриптов осуществляется автоматически через `spring.jpa.hibernate.ddl-auto=update`.

Таблица 11. Описание таблиц базы данных SPSHOP

№	Таблица	Назначение	Ключевые поля
1	user	Аккаунты пользователей и администраторов	id, name, email, password (BCrypt), role, isEnabled, accountStatusNonLocked, accountfailedAttemptCount, accountLockTime, resetTokens
2	product	Каталог товаров спортивного питания	id, title, description, price, discountPrice, discount, stock, image, isActive, category_id
3	category	Категории товаров	id, name, imageUrl, isActive
4	cart	Корзина покупателя	id, user_id, product_id, quantity
5	product_order	Оформленные заказы	id, orderId (UUID), orderDate, price, quantity, status, paymentType, transactionId (зашифр.), user_id, product_id
6	order_address	Адрес доставки для заказа	id, firstName, lastName, email, mobile, address, city, state, pinCode, omnivaTerminal, smartpostTerminal

Связи между таблицами:

- `user` → `cart` : OneToMany (один пользователь — много позиций в корзине)
- `product` → `cart` : ManyToOne (один товар может быть в нескольких корзинах)
- `category` → `product` : OneToMany (одна категория — много товаров)
- `user` → `product\_order` : OneToMany (один пользователь — много заказов)
- `product\_order` → `order\_address` : OneToOne (один заказ — один адрес)

ER-диаграмма связей базы данных представлена ниже:

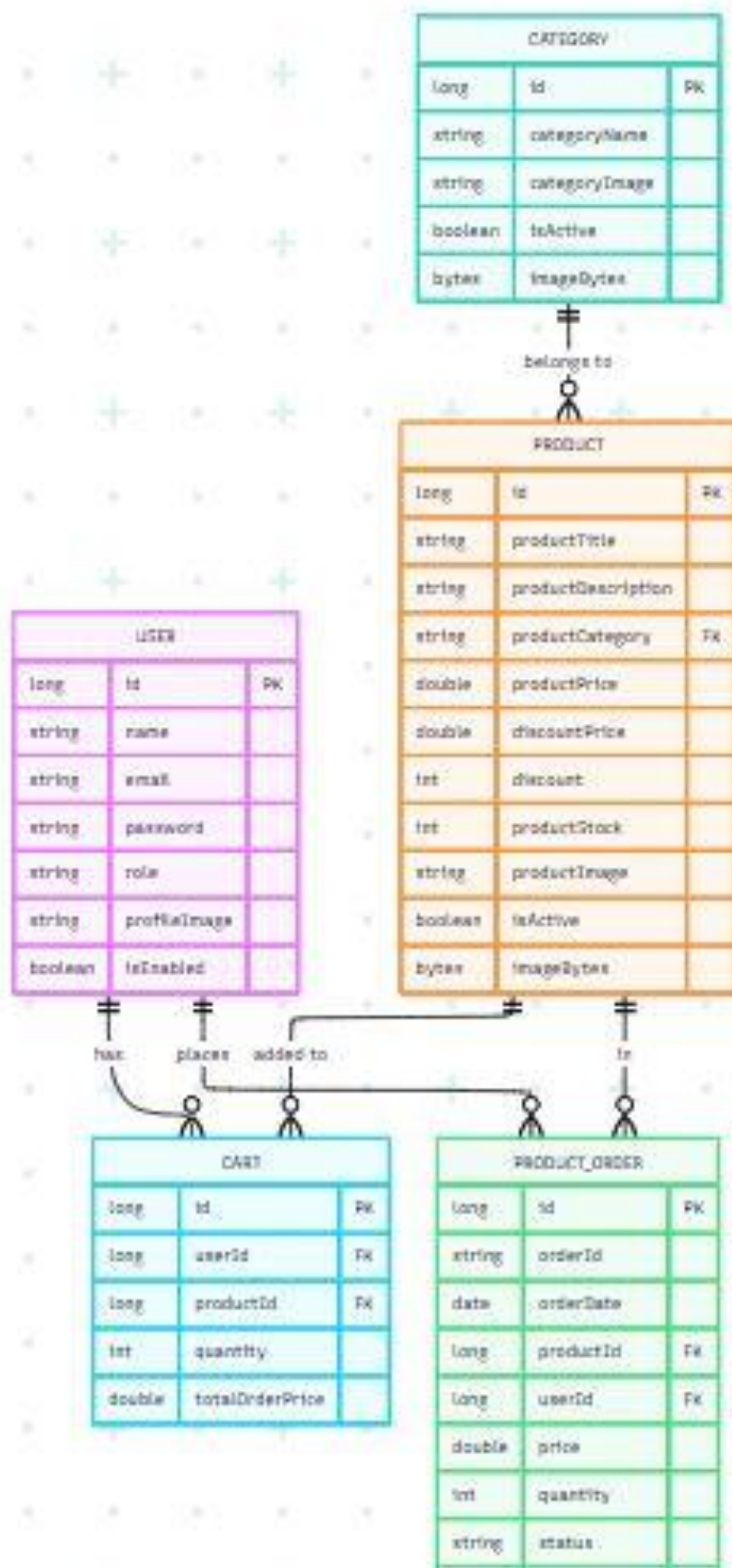


Схема структуры таблиц и их связей, импортированная из СУБД в MySQL Workbench:

3.3.3. Права доступа и безопасность (Security & RBAC)

Настройка безопасности реализована в классе `SecurityConfig`, аннотированном `@EnableWebSecurity`. Используется подход **RBAC (Role-Based Access Control)** — управление доступом на основе ролей. В системе определены две роли: `ROLE_USER` (покупатель) и `ROLE_ADMIN` (администратор).

Конфигурация цепочки фильтров безопасности (`filterChain`) определяет следующие правила доступа к эндпоинтам:

- Публичные маршруты (доступны всем без авторизации): `/`, `/shop/`, `/product/`, `/user/register`, `/user/forgot-password`, `/user/reset-password`, `/img/`, `/js/`, `/css/`.
- Маршруты покупателя (требуют роль `ROLE_USER`): `/user/` — корзина, оформление заказа, оплата, личный кабинет.
- Маршруты администратора (требуют роль `ROLE_ADMIN`): `/admin/` — управление товарами, категориями, заказами, пользователями.
- REST API оплаты `/api/payment/` — доступ только для аутентифицированных пользователей.

Настройка страниц входа и выхода:

- Страница входа: `/login` с параметром `?error` при неверных данных.
- После успешного входа: администратор перенаправляется на `/admin/`, покупатель — на `/user/`.
- Выход из системы: POST-запрос на `/logout` с очисткой сессии и cookie.

Ниже представлен скриншот конфигурации цепочки фильтров безопасности:

```

1  @Bean
2  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
3      http.csrf(csrf -> csrf.disable())
4      .cors(cors->cors.disable())
5      .authenticationProvider(authenticationProvider())
6      .authorizeHttpRequests(req-> req.requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
7      .requestMatchers("/admin/**").hasRole("ADMIN")
8      .requestMatchers("/**").permitAll())
9      .formLogin(form-> form.loginPage("/signin")
10         .loginProcessingUrl("/login")
11         //defaultSuccessUrl("/")//before implements authenticationSuccessHandler.
12         //after implementation authenticationSuccessHandler -> call successHandler
13         .failureHandler(authenticationFailureHandler)
14         .successHandler(authenticationSuccessHandler))
15
16     .logout(logout->logout.permitAll());
17     return http.build();
18
19 }

```

Рисунок. Настройка цепочки фильтров безопасности Spring Security

Утилита SecurityUtils предоставляет функции шифрования и расшифровки номеров транзакций Stripe перед их сохранением в БД. Это предотвращает утечку финансовых идентификаторов в случае компрометации СУБД. Шифрование выполняется через алгоритм `AES/CBC/PKCS5Padding` с секретным ключом из переменных окружения.

Защита от CSRF обеспечивается встроенным механизмом Spring Security. Каждый POST-запрос формы должен содержать валидный CSRF-токен, который Thymeleaf автоматически добавляет в форму через атрибут `th:action`.

3.3.4. Контроллеры и API

Веб-приложение содержит следующие группы контроллеров, разделённых по зонам ответственности:

Таблица 12. Контроллеры приложения SPSHOP

Контроллер	Маппинг	Тип	Назначение
`HomeController`	`/`	MVC	Главная страница, каталог, поиск, детали товара, регистрация, вход, сброс пароля
`UserController`	`/user`	MVC	Личный кабинет, корзина,

			оформление заказа, история покупок
`AdminController`	`/admin`	MVC	Панель администратора, управление товарами, заказами, пользователями
`CartRestController`	`/api/cart`	REST	AJAX-запросы: добавление, обновление количества, удаление из корзины
`PaymentRestController`	`/api/payment`	REST	Создание PaymentIntent через Stripe API, обработка webhook
`ImageController`	`/image`	REST	Отдача изображений товаров из файловой системы
`TestVideoController`	`/test-videos`	MVC	Просмотр видеозаписей Playwright-тестов

Метод ``@ModelAttribute`` в классе ``UserController`` привязывается к каждому запросу пользователя и автоматически передает в шаблон данные текущего аутентифицированного пользователя, количество товаров в корзине и список активных категорий:

```
@ModelAttribute
public void getUserDetails(Principal principal, Model model) {
    if (principal != null) {
```

```

        User currentUserDetails = userService.getUserByEmail(principal.getName());
        model.addAttribute("currentLoggedInUserDetails", currentUserDetails);
        Long countCartForUser =
        cartService.getCounterCart(currentUserDetails.getId());
        model.addAttribute("countCartForUser", countCartForUser);
    }
    model.addAttribute("allActiveCategory", categoryService.findAllActiveCategory());
}

```

3.3.5. Реализация авторизации, регистрации и сессий

Регистрация нового пользователя реализована через метод `saveUser()` в классе `UserServiceImpl`. При регистрации система выполняет следующие действия:

1. Устанавливает роль `ROLE\_USER` (все новые аккаунты — покупатели).
2. Активирует аккаунт: `isEnabled = true`.
3. Устанавливает статус незаблокированного: `accountStatusNonLocked = true`.
4. Обнуляет счетчик попыток: `accountFailedAttemptCount = 0`.
5. Хеширует пароль через BCryptPasswordEncoder:
`passwordEncoder.encode(user.getPassword())`.
6. Сохраняет пользователя в базу данных через `UserRepository`.

Ниже представлен скриншот реализации метода регистрации нового пользователя:

```

1      @Override
2      public User saveUser(User user) {
3          System.out.println("user obje :"+user.toString());
4          user.setRole("ROLE_USER");
5          user.setIsEnable(true);
6          user.setAccountStatusNonLocked(true);
7          user.setAccountFailedAttemptCount(0);
8          user.setAccountLockTime(null);
9          String encodedPassword = passwordEncoder.encode(user.getPassword());
10         user.setPassword(encodedPassword);
11         try {
12             User saveUser = userRepository.save(user);
13
14             return saveUser;
15         } catch (Exception e) {
16             throw new RuntimeException("Failed to create user", e);
17         }
18     }

```

Рисунок. Реализация функции регистрации пользователя

Система блокировки аккаунтов. При каждом неудачном входе срабатывает ``AuthFailureHandlerImpl`` — кастомный обработчик события ``AuthenticationFailureHandler``. Алгоритм работы:

1. По email находится пользователь в БД.
2. Увеличивается счетчик ``accountfailedAttemptCount`` на 1.
3. Если счетчик достиг 3 — вызывается ``userAccountLock()``: флаг ``accountStatusNonLocked`` устанавливается в ``false``, фиксируется ``accountLockTime = new Date()``.
4. При следующей попытке входа Spring Security проверяет флаг через ``UserDetailsService``. Если аккаунт заблокирован, проверяется время блокировки: если прошло 24 часа (``AppConstant.UNLOCK_DURATION_TIME``), аккаунт разблокируется автоматически через ``isUnlockAccountTimeExpired()``.

Загрузка данных пользователя для Spring Security выполняется через ``UserDetailsServiceImpl``, который реализует интерфейс ``UserDetailsService``. Метод ``loadUserByUsername()`` загружает пользователя по email из репозитория и создаёт объект ``UserDetails`` с учётом статуса блокировки аккаунта.

```
1  @Override
2  public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
3      System.out.println("username " + username);
4      User user = userRepository.findByEmail(username);
5      if(user == null) {
6          throw new UsernameNotFoundException("User NOT Found for :"+username);
7      }
8      return new CustomUser(user);
9  }
```

Рисунок. Реализация сервиса загрузки пользователей UserDetailsService

Для представления учетной записи пользователя в базе данных и маппинга JPA-сущности на таблицу ``user`` спроектирована модель ``User``:

```
1  @Entity
2  @AllArgsConstructor
3  @NoArgsConstructor
4  @Setter
5  @Getter
6  @ToString
7  public class User {
8
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long id;
12
13     private String name;
14
15     private String mobile;
16
17     private String email;
18
19     private String password;
20
21     private String profileImage;
22
23     private String role;
24
25     private Boolean isEnabled;
26
27     //implement user account lock for wrong password
28     private Boolean accountStatusNonLocked;
29
30     private Integer accountFailedAttemptCount;
31
32     private Date accountLockTime;
33
34     private String resetTokens;
35
36     @CreationTimestamp
37     private LocalDateTime createdAt;
38
39     @UpdateTimestamp
40     private LocalDateTime updatedAt;
41
```

Рисунок. Код сущности User JPA Entity

3.3.6. Разработка Frontend

Фронтенд реализован на базе **HTML5** с использованием шаблонизатора **Thymeleaf**. Все шаблоны разделены на переиспользуемые фрагменты через механизм ``th:fragment``:

- ``fragments/header.html`` — шапка с навигацией, строкой поиска, значком корзины.
- ``fragments/footer.html`` — подвал с контактами и ссылками.
- ``fragments/sidebar.html`` — боковое меню с категориями для административной панели.

Интерфейс гостевой страницы каталога товаров спортивного питания представлен ниже:

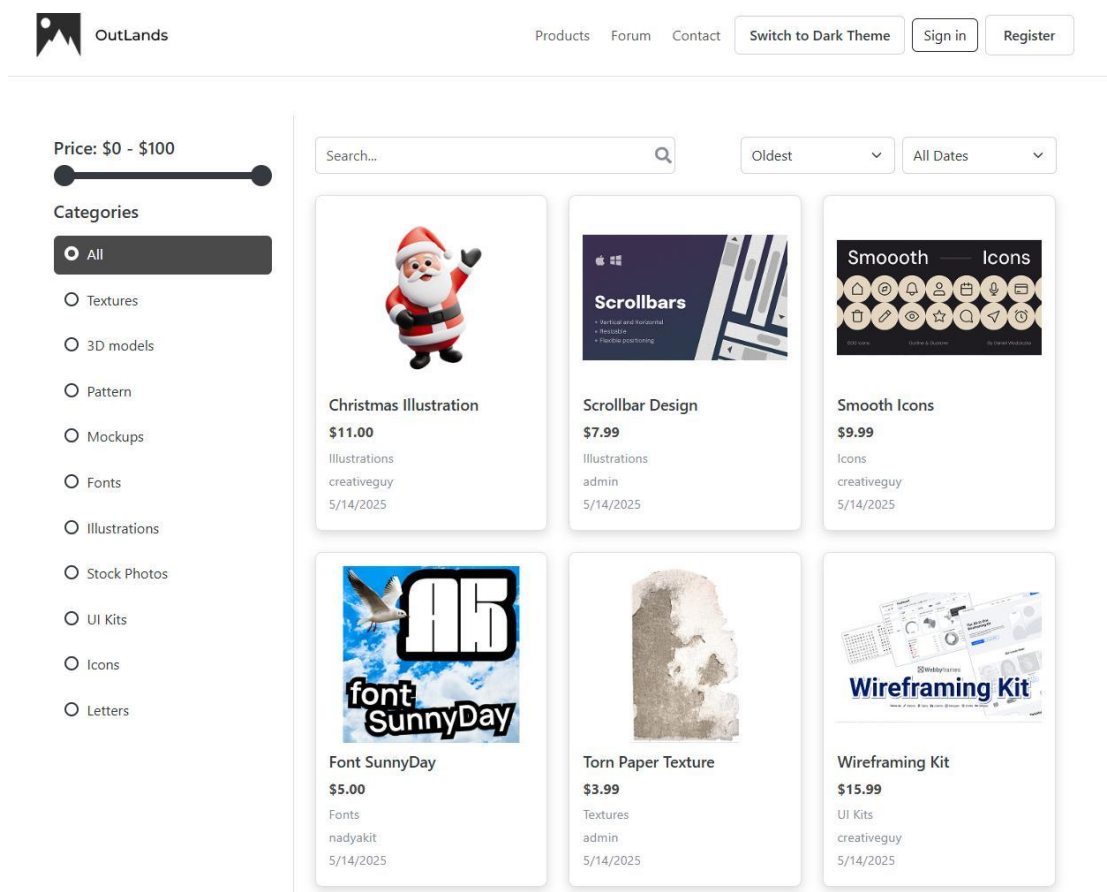


Рисунок. Каталог товаров для гостей

Для входа и регистрации пользователей спроектированы формы авторизации и создания аккаунта:

Интерфейс личного кабинета зарегистрированного пользователя (Клиента) с историей его заказов:

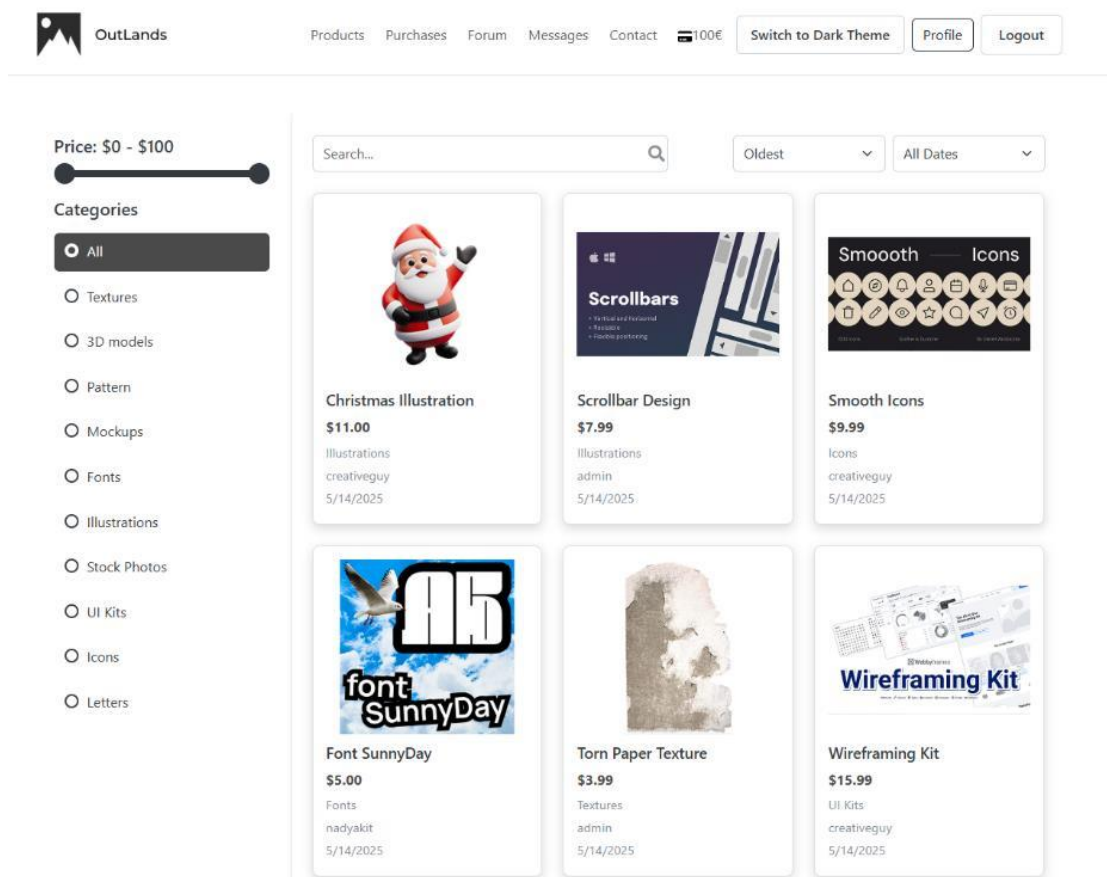


Рисунок. Профиль зарегистрированного пользователя

Технологии фронтенда:

- Thymeleaf — динамическая вставка данных из модели (товары, категории, корзина, пользователь) в HTML-шаблоны.
- Vanilla CSS с переменными `--primary-color`, `--accent-color`, `--bg-dark` — единая дизайн-система.
- CSS backdrop-filter — глассморфизм (эффект матового стекла на карточках).
- JavaScript Fetch API — асинхронные запросы к REST-контроллерам (добавление в корзину, Stripe payment intent).
- Stripe.js / Stripe Elements — безопасная форма ввода данных карты в iframe.

Адаптивность реализована через CSS медиа-запросы (`@media (max-width: 768px)`), обеспечивающие корректное отображение на экранах мобильных устройств:

одноколоночная сетка каталога, скрытое боковое меню, адаптированные кнопки.

Фронтенд-логика Stripe (инициализация, создание формы, обработка платежа)
представлена на скриншоте ниже:

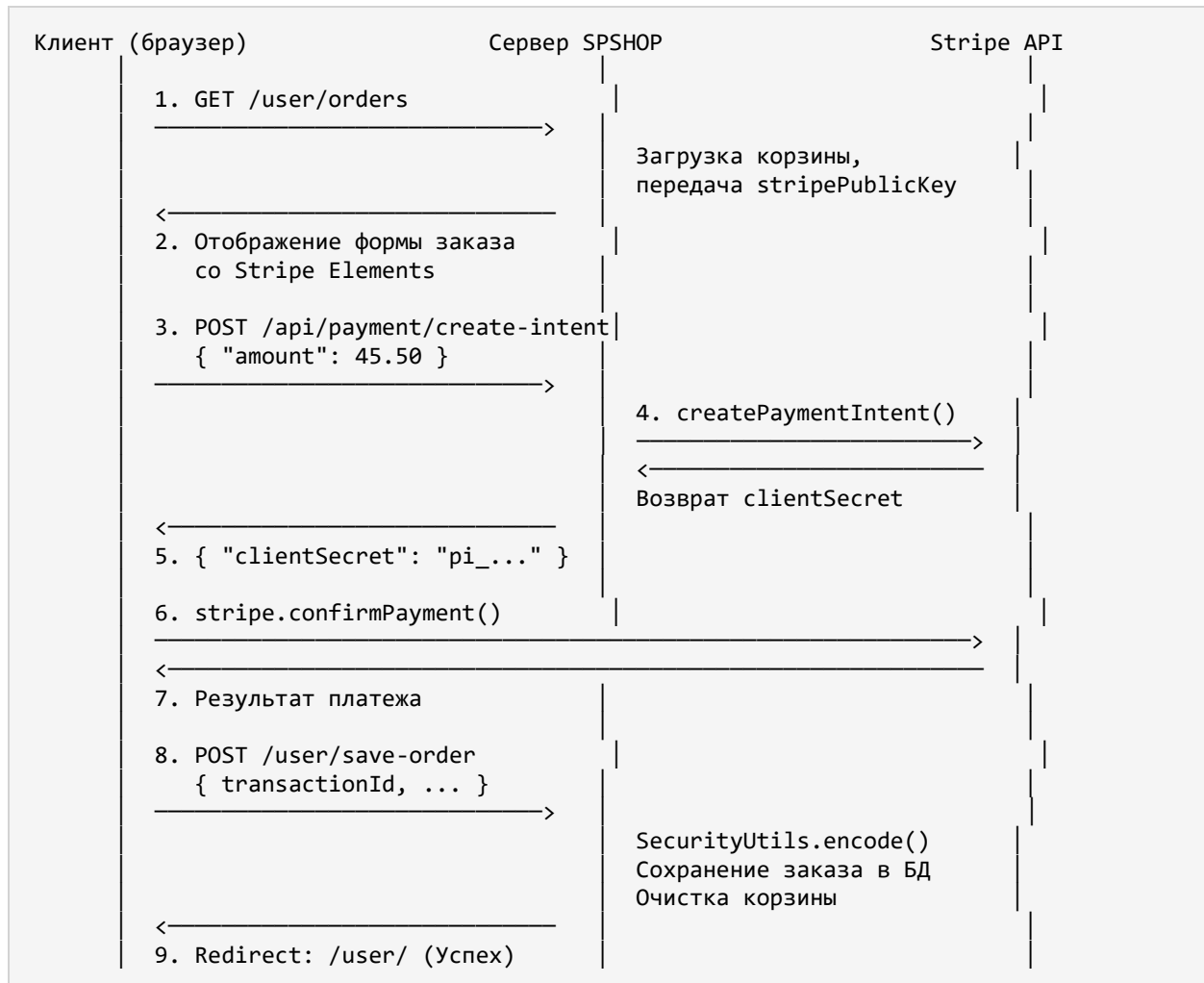
```
1  async function startStripePayment(form) {
2    Swal.fire({
3      title: 'Preparing Secure Payment...',
4      didOpen: () => { Swal.showLoading(); },
5      background: '#0a1226',
6      color: '#fff',
7      allowOutsideClick: false
8    });
9
10   try {
11     const response = await fetch('/api/payment/create-intent', {
12       method: 'POST',
13       headers: { 'Content-Type': 'application/json' },
14       body: JSON.stringify({ amount: [[${totalOrderPrice}]] });
15     });
16     const { clientSecret, error } = await response.json();
17
18     if (error) throw new Error(error);
19
20     showStripeModal(clientSecret, form);
21   } catch (err) {
22     Swal.fire({ icon: 'error', title: 'Payment Error', text: err.message, background: '#0a1226', color: '#fff' });
23   }
24 }
25
26 function showStripeModal(clientSecret, form) {
27   Swal.fire({
28     title: 'SPSHOP Secure Payment',
29     html: `
30       <div class="text-start mt-3">
31         <div class="p-3 mb-3" style="background: rgba(255,255,255,0.05); border-radius: 12px; border: 1px solid rgba(255,255,255,0.1);">
32           <p class="small text-white-50 mb-1">Total Amount</p>
33           <h3 class="text-warning mb-0">&euro;[[${totalOrderPrice}]]</h3>
34         </div>
35         <div id="payment-element"></div>
36         <div id="payment-message" class="text-danger small mt-2 d-none"></div>
37       </div>
38     `,
39     showCancelButton: true,
40     confirmButtonText: 'Pay Now',
41     cancelButtonText: 'Cancel',
42     confirmButtonColor: '#daa520',
43     cancelButtonColor: '#30363d',
44     background: '#0a1226',
45     color: '#fff',
46     customClass: { popup: 'glass-card border-0 shadow-lg', confirmButton: 'text-dark fw-bold px-4' },
47     didOpen: () => {
48       const appearance = { theme: 'night', variables: { colorPrimary: '#daa520' } };
49       elements = stripe.elements({ appearance, clientSecret });
50       const paymentElement = elements.create("payment");
51       paymentElement.mount("#payment-element");
52     },
53     preConfirm: async () => {
54       Swal.showLoading();
55       const { paymentIntent, error } = await stripe.confirmPayment({
56         elements,
57         confirmParams: { return_url: window.location.origin + "/user/orders" },
58         redirect: 'if_required'
59       });
60
61       if (error) {
62         const messageContainer = document.querySelector("#payment-message");
63         messageContainer.textContent = error.message;
64         messageContainer.classList.remove('d-none');
65         Swal.hideLoading();
66         return false;
67       }
68       return paymentIntent; // Pass paymentIntent to result
69     }
70   }).then((result) => {
71     if (result.isConfirmed) {
72       Swal.fire({
73         title: 'Payment Successful!',
74         text: 'Redirecting to your orders...',
75         icon: 'success',
76         showConfirmButton: false,
77         timer: 1500,
78         background: '#0a1226',
79         color: '#fff'
80       }).then(() => {
81         // Set the transaction ID before submitting
82         if (result.value && result.value.id) {
83           document.getElementById('transactionIdInput').value = result.value.id;
84         }
85         form.submit();
86       });
87     }
88   });
89 }
```

Рисунок. Фронтенд-интеграция Stripe Elements через JavaScript

3.3.7. Интеграция с платежным шлюзом Stripe

Интеграция с **Stripe** реализована с использованием официального **Stripe Java SDK версии 24.0.0**. Используется подход **PaymentIntent API** — современная и наиболее безопасная модель обработки платежей, рекомендованная Stripe для серверного подтверждения.

Схема процесса оплаты:



Серверная часть (StripeService):

Класс `StripeService` инициализирует Stripe SDK ключом из переменных окружения через аннотацию `@Value("${stripe.api.secretKey}")`. Метод `createPaymentIntent()` принимает сумму в евро, конвертирует её в центы (умножение на 100 — требование Stripe API),

создаёт объект `PaymentIntentCreateParams` с включёнными автоматическими методами оплаты и вызывает `PaymentIntent.create()`:

```
@Service
public class StripeService {

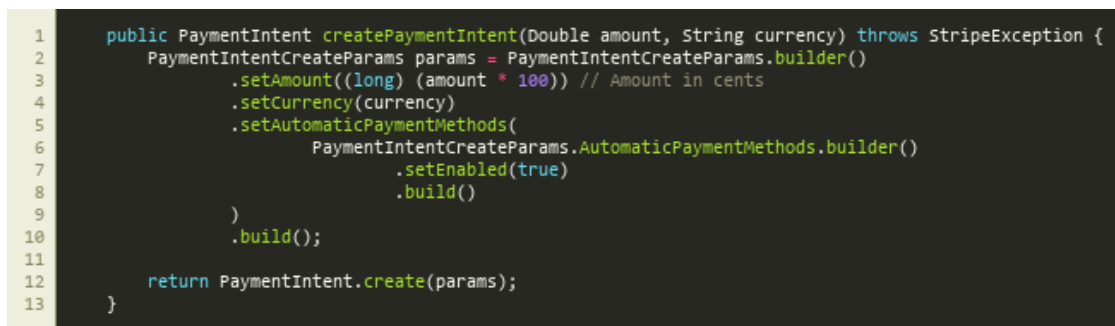
    @Value("${stripe.api.secretKey}")
    private String secretKey;

    @PostConstruct
    public void init() {
        Stripe.apiKey = secretKey;
    }

    public PaymentIntent createPaymentIntent(Double amount, String currency)
        throws StripeException {
        PaymentIntentCreateParams params = PaymentIntentCreateParams.builder()
            .setAmount((long) (amount * 100)) // Конвертация евро → центы
            .setCurrency(currency)           // "eur"
            .setAutomaticPaymentMethods(
                PaymentIntentCreateParams.AutomaticPaymentMethods.builder()
                    .setEnabled(true)
                    .build()
            )
            .build();
        return PaymentIntent.create(params);
    }
}
```

Метод `@PostConstruct init()` вызывается один раз при запуске контейнера Spring и устанавливает глобальный API-ключ для всех последующих вызовов Stripe SDK.

Скриншот реализации `StripeService`:

A screenshot of a code editor showing the implementation of the `createPaymentIntent` method in the `StripeService` class. The code is written in Java and includes comments in Russian. The method takes a `Double amount` and a `String currency` as input and returns a `PaymentIntent` object. It uses the Stripe SDK's builder pattern to create a `PaymentIntentCreateParams` object, setting the amount in cents, the currency, and enabling automatic payment methods. The code is highlighted with a dark background and light text, with line numbers on the left.

```
1 public PaymentIntent createPaymentIntent(Double amount, String currency) throws StripeException {
2     PaymentIntentCreateParams params = PaymentIntentCreateParams.builder()
3         .setAmount((long) (amount * 100)) // Amount in cents
4         .setCurrency(currency)
5         .setAutomaticPaymentMethods(
6             PaymentIntentCreateParams.AutomaticPaymentMethods.builder()
7                 .setEnabled(true)
8                 .build()
9         )
10        .build();
11
12    return PaymentIntent.create(params);
13 }
```

Рисунок. Создание платежного намерения Stripe `PaymentIntent` на бэкенде

REST-контроллер оплаты (`PaymentRestController`):

Endpoint `POST /api/payment/create-intent` принимает JSON с суммой заказа, делегирует создание `PaymentIntent` в `StripeService` и возвращает клиенту `clientSecret` — уникальный ключ для подтверждения платежа на стороне браузера:

```
@RestController
@RequestMapping("/api/payment")
public class PaymentRestController {

    @Autowired
    private StripeService stripeService;

    @PostMapping("/create-intent")
    public ResponseEntity<Map<String, String>> createPaymentIntent(
        @RequestBody Map<String, Object> data) {
        try {
            Double amount = Double.parseDouble(data.get("amount").toString());
            String currency = "eur"; // По умолчанию евро для SPSHOP
            PaymentIntent intent = stripeService.createPaymentIntent(amount,
currency);
            Map<String, String> response = new HashMap<>();
            response.put("clientSecret", intent.getClientSecret());
            return ResponseEntity.ok(response);
        } catch (StripeException | NumberFormatException e) {
            Map<String, String> error = new HashMap<>();
            error.put("error", e.getMessage());
            return ResponseEntity.badRequest().body(error);
        }
    }
}
```

Скриншот кода REST-контроллера оплаты:

```
1  @PostMapping("/create-intent")
2  public ResponseEntity<Map<String, String>> createPaymentIntent(@RequestBody Map<String, Object> data) {
3      try {
4          Double amount = Double.parseDouble(data.get("amount").toString());
5          String currency = "eur"; // Default to Euro for SPSHOP
6
7          PaymentIntent intent = stripeService.createPaymentIntent(amount, currency);
8
9          Map<String, String> response = new HashMap<>();
10         response.put("clientSecret", intent.getClientSecret());
11
12         return ResponseEntity.ok(response);
13     } catch (StripeException | NumberFormatException e) {
14         Map<String, String> error = new HashMap<>();
15         error.put("error", e.getMessage());
16         return ResponseEntity.badRequest().body(error);
17     }
18 }
```

Рисунок. REST-контроллер обработки платежей

Страница оформления заказа и Stripe Elements:

При открытии страницы `/user/orders` контроллер `UserController` передаёт в шаблон:

- Список позиций корзины (`carts`).
- Цену заказа (`orderPrice`) — сумма товаров.
- Итоговую цену с доставкой (`totalOrderPrice = orderPrice + 250 + 100`) — включает стоимость упаковки и доставки в центах.
- Публичный ключ Stripe (`stripePublicKey`) из переменной окружения.

На странице оформления заказа JavaScript-код инициализирует **Stripe Elements** с использованием публичного ключа. Пользователь вводит данные карты в изолированный iframe Stripe — данные карты никогда не передаются через серверы SPSHOP (соответствие требованиям PCI DSS SAQ A).

Процесс на стороне фронтенда:

1. JavaScript отправляет AJAX-запрос на `/api/payment/create-intent` с суммой заказа.
2. Получает `clientSecret` из ответа сервера.
3. Передаёт `clientSecret` в `stripe.confirmPayment()` вместе с данными из Stripe Elements формы.
4. Stripe проводит платёж и возвращает результат.
5. При успехе JavaScript отправляет POST на `/user/save-order` с данными формы и `transactionId`.

Сохранение заказа и шифрование транзакции:

В методе `saveProductOrder()` сервиса `ProductOrderServiceImpl` для каждой позиции корзины создаётся отдельная запись `ProductOrder`. Уникальный идентификатор заказа генерируется через `UUID.randomUUID()`. Идентификатор транзакции Stripe шифруется через `SecurityUtils.encodePaymentId()` перед сохранением в БД:

```
order.setOrderId(UUID.randomUUID().toString());
order.setTransactionId(SecurityUtils.encodePaymentId(req.getTransactionId()));
order.setStatus("Success");
```

После сохранения всех позиций заказа корзина полностью очищается через `cartRepository.deleteAll(listOfCarts)`.

Конфигурация Stripe в application.properties:

Секретный и публичный ключи Stripe не хранятся в репозитории и передаются через переменные окружения:

```
stripe.api.secretKey=${STRIPE_SECRET_KEY}
stripe.api.publicKey=${STRIPE_PUBLIC_KEY}
```

При локальной разработке используются тестовые ключи вида `sk\_test\_...` и `pk\_test\_...`, предоставляемые Stripe в панели разработчика. Для тестирования платежей используются специальные тестовые номера карт Stripe (например, `4242 4242 4242 4242` для успешного платежа).

3.3.8. Корзина товаров и управление заказами

Сервис корзины (**CartServiceImpl**) реализует три основных операции:

Добавление товара в корзину — метод `saveCart(Long productId, Long userId)`:

1. Загружает пользователя и продукт из репозитория.
2. Проверяет, существует ли уже запись в корзине для данной пары пользователь-товар.
3. Если записи нет — создаёт новую запись Cart с количеством 1 и ценой равной `discountPrice`.
4. Если запись уже есть — увеличивает количество на 1 и пересчитывает `totalPrice = quantity \* discountPrice`.

Получение корзины пользователя — метод `getCartsByUser(Long userId)`:

1. Загружает все позиции корзины для пользователя.
2. Для каждой позиции вычисляет `totalPrice = discountPrice \* quantity` (поле @Transient не хранится в БД).
3. Аккумулирует `totalOrderPrice` — накопленную сумму всех позиций.
4. Возвращает обновленный список позиций.

Обновление количества — метод `updateCartQuantity(String symbol, Long cartId)`:

- `symbol = "increase"` — увеличивает количество на 1.
- `symbol = "decrease"` — уменьшает количество на 1. Если количество стало ≤ 0 , позиция удаляется из корзины.

Скриншот реализации корзины:

```

1  @Override
2  public Cart saveCart(Long productId, Long userId) {
3
4      User user = userRepository.findById(userId).get();
5      Product product = productRepository.findById(productId).get();
6
7      Cart cartStatus = cartRepository.findByProductIdAndUserId(productId, userId);
8
9      Cart cart = null;
10
11     if(ObjectUtils.isEmpty(cartStatus)) {
12         //cart is empty - so add product to cart
13         cart = new Cart();
14         cart.setUser(user);
15         cart.setProduct(product);
16         cart.setQuantity(1);
17         cart.setTotalPrice(1 * product.getDiscountPrice());
18     }else {
19         //cart is holding product. so increases the quantity of the existing product.
20         cart = cartStatus;
21         cart.setQuantity(cart.getQuantity()+1);
22         cart.setTotalPrice(cart.getQuantity() * cart.getProduct().getDiscountPrice());
23     }
24     Cart saveCart = cartRepository.save(cart);
25
26     return saveCart;
27 }

```

Рисунок. Сервис управления корзиной покупок

Реализация логики оформления и сохранения заказов представлена в сервисе
`ProductOrderServiceImpl`:


```

1  @Override
2  public ProductOrder saveProductOrder(Long userId, ProductOrderRequest req) {
3
4      List<Cart> listofCarts = cartRepository.findByUserId(userId);
5      ProductOrder finalOrder = null;
6
7      for (Cart cart : listofCarts) {
8
9          // Build shipping address
10         OrderAddress address = new OrderAddress();
11         address.setFirstName(req.getFirstName());
12         address.setLastName(req.getLastName());
13         address.setEmail(req.getEmail());
14         address.setMobile(req.getMobile());
15         address.setAddress(req.getAddress());
16         address.setCity(req.getCity());
17         address.setState(req.getState());
18         address.setPinCode(req.getPinCode());
19
20         // Build order
21         ProductOrder order = new ProductOrder();
22         order.setOrderId(UUID.randomUUID().toString());
23         order.setOrderDate(new Date());
24         order.setProduct(cart.getProduct());
25         order.setPrice(cart.getProduct().getDiscountPrice());
26         order.setQuantity(cart.getQuantity());
27         order.setUser(cart.getUser());
28
29         // Handle Payment Status and Transaction ID
30         if ("Online Payment".equalsIgnoreCase(req.getPaymentType())) {
31             order.setStatus("Success");
32             // Encode the transaction ID as requested (2jwt style)
33             order.setTransactionId(SecurityUtils.encodePaymentId(req.getTransactionId()));
34         } else {
35             order.setStatus("In Progress");
36         }
37
38         order.setPaymentType(req.getPaymentType());
39         order.setOrderAddress(address);
40
41         finalOrder = productOrderRepository.save(order);
42     }
43
44     // Clear cart after placing order
45     cartRepository.deleteAll(listofCarts);
46
47     return finalOrder;
48 }

```

Рисунок. Реализация сервиса оформления заказов

REST-контроллер корзины (CartRestController):

Обрабатывает AJAX-запросы с фронтенда для обновления количества товаров без перезагрузки страницы. Возвращает JSON-ответ с обновлённым итогом корзины, который JavaScript использует для обновления DOM без полного перерендеринга страницы.

3.3.9. Интеграция с почтовыми отправлениями (Omniva и SmartPost)

На территории Эстонии действуют две крупнейших сети почтовых терминалов самообслуживания: **Omniva** (государственная почтовая служба) и **SmartPost** (частный

оператор). Для удобства покупателей в форму оформления заказа интегрирован выбор конкретного терминала из выпадающего меню.

Реализация:

- База данных содержит предзагруженные списки эстонских городов и привязанных к ним терминалов каждого провайдера.
- В форме оформления заказа пользователь сначала выбирает провайдера (Omniva или SmartPost), затем из динамически обновляемого выпадающего меню — конкретный терминал в своём городе.
- Выбранный терминал сохраняется в полях `omnivaTerminal` и `smartpostTerminal` сущности `OrderAddress`, связанной с заказом.
- Администратор видит выбранный терминал в деталях заказа при обработке отправления.

Расчёт стоимости доставки выполняется на странице `/user/orders`:

```
Double totalOrderPrice = orderPrice + 250 + 100;  
// 250 – стоимость упаковки (центы), 100 – стоимость доставки (центы)
```

Это позволяет включить расходы на доставку в итоговую сумму при создании PaymentIntent.

Таблица 13. Поддерживаемые почтовые службы

Параметр	Omniva	SmartPost
Тип	Государственная почта Эстонии	Частный оператор DPD
Покрытие	По всей Эстонии	По всей Эстонии
Режим работы	24/7	24/7
Идентификация	Код на телефоне	Код на телефоне
Поле в БД	omnivaTerminal	smartpostTerminal

3.3.10. Панель администратора

Административная панель доступна по адресу `/admin/` и защищена ролью `ROLE\_ADMIN`. Контроллер `AdminViewController` обслуживает все маршруты административной зоны.

Интерфейс контрольной панели администратора (дашборд):

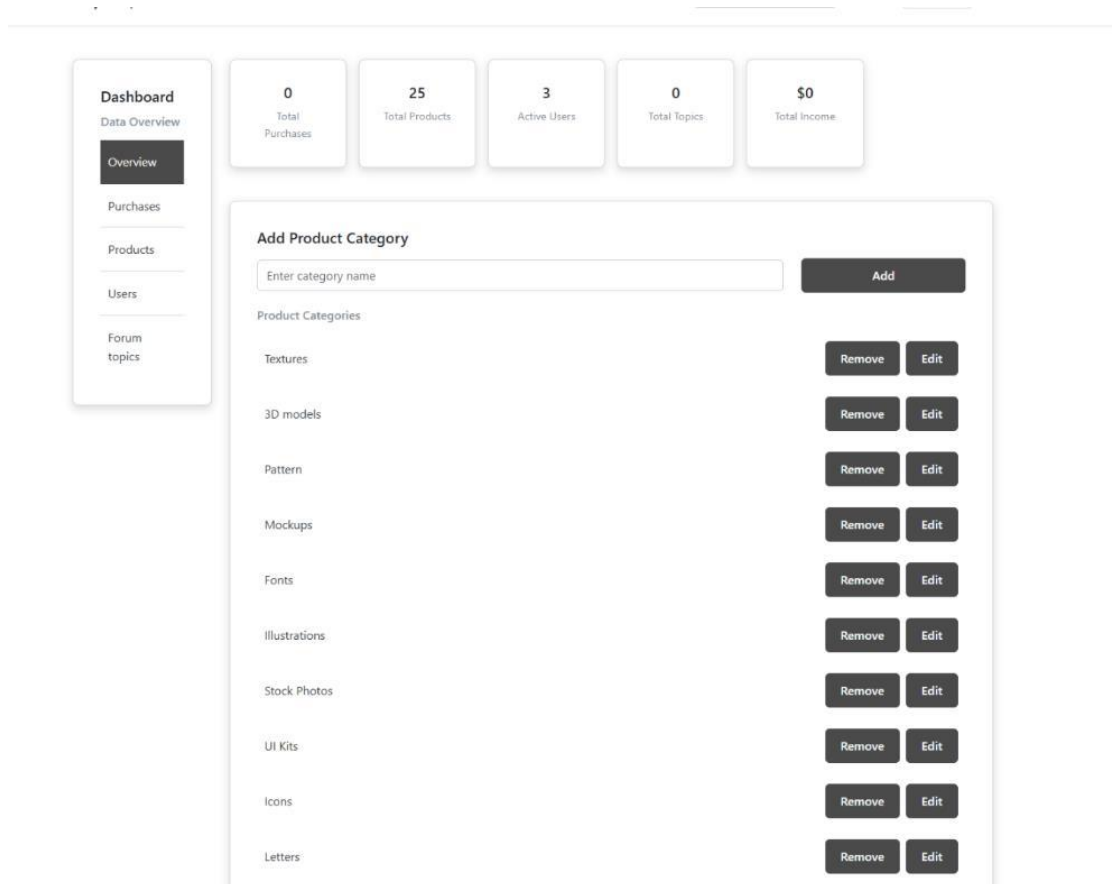


Рисунок. Панель управления администратора

Компоненты административной панели:

1. Дашборд (`/admin/`) — сводная аналитика: общее количество пользователей-покупателей, активных заказов, товаров в каталоге, категорий. Данные отображаются в виде карточек с иконками.
2. Управление категориями (`/admin/category`) — CRUD-операции для категорий спортивного питания. Администратор может добавлять новые категории с изображениями, деактивировать неиспользуемые (деактивированные категории не отображаются покупателям).

3. Управление товарами (`/admin/product`) — полный CRUD для продуктов. При добавлении администратор указывает: название, описание, цену, процент скидки (автоматически вычисляется `discountPrice`), остаток на складе (`stock`), изображение, категорию и статус активности.
4. Управление заказами (`/admin/order`) — список всех заказов с возможностью изменения статуса. Доступные статусы: `Ordered` → `Payment Approved` → `Shipped` → `Delivered` → `Cancelled`. Администратор видит зашифрованный ID транзакции Stripe для каждого онлайн-заказа.
5. Управление пользователями (`/admin/user`) — список всех покупателей системы. Отображается статус аккаунта. Заблокированные аккаунты выделены визуально. Администратор может разблокировать аккаунт кнопкой «Unlock».

3.3.11. Развертывание на хостинге

Платформа развернута на облачной инфраструктуре **Microsoft Azure**.

Скриншот статуса развертывания веб-приложения на хостинге Azure:

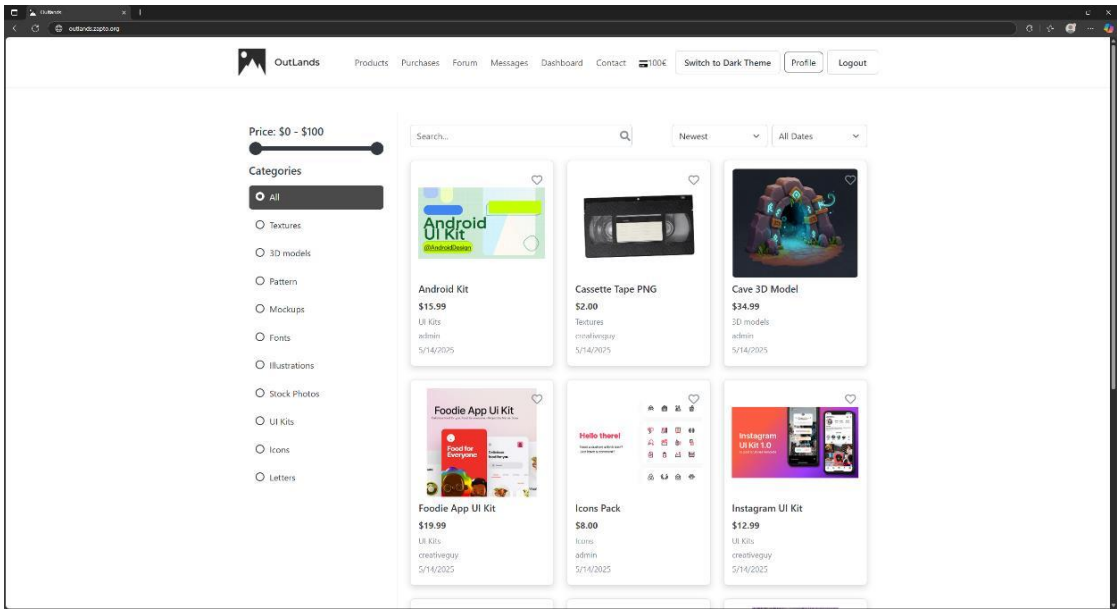


Рисунок. Статус развертывания на Azure

Таблица 14. Конфигурация производственной среды

Компонент	Технология	Конфигурация
Веб-сервер	Azure App Service	Linux контейнер, B1 (1 vCPU, 1.75 GB RAM)

Контейнер приложения	Docker	openjdk:17-jdk-slim базовый образ
База данных	Azure Database for MySQL	Flexible Server, 5.0 GB хранилище
SSL/HTTPS	Azure App Service SSL	Автоматический managed certificate
Конфигурация	Azure Environment Variables	Stripe ключи, DB credentials

Процесс развёртывания:

1. Сборка JAR-файла: ``mvnw package -DskipTests``.
2. Сборка Docker-образа: ``docker build -t spshop:latest .``.
3. Push в Azure Container Registry.
4. Azure App Service автоматически подтягивает новый образ и перезапускает контейнер.
5. При старте Hibernate автоматически применяет изменения схемы БД (``ddl-auto=update``).

Dockerfile использует многоступенчатую сборку для минимизации размера финального образа и использует непривилегированного пользователя для запуска JVM в целях безопасности.

4. ТЕСТИРОВАНИЕ ПРОЕКТА И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1. Выбор методов тестирования

Для обеспечения надежности платформы SPSHOP были выбраны методы автоматизированного тестирования трёх уровней, что соответствует концепции «пирамиды тестирования»:

1. Модульное тестирование (Unit Testing) — Белый ящик: Изолированная проверка отдельных методов бизнес-логики с использованием JUnit 5 и Mockito. Зависимости (репозитории, другие сервисы) заменяются заглушками (mocks). Охватывает сервисный слой: ``UserServiceImpl``, ``CartServiceImpl``, ``ProductServiceImpl``, ``ProductOrderServiceImpl``, ``CategoryServiceImpl``.

2. Интеграционное тестирование — Белый ящик: Проверка взаимодействия контроллеров с бизнес-логикой с использованием MockMvc. Тесты выполняются с in-memory базой данных H2, которая эмулирует MySQL без необходимости внешнего сервера БД. Это обеспечивает полную независимость тестов от среды разработчика.

3. Сквозное E2E тестирование — Черный ящик: Автоматические браузерные сценарии с использованием Microsoft Playwright. Playwright запускает реальный headless-браузер Chromium и имитирует действия живого пользователя: открытие страниц, заполнение форм, навигация, верификация состояния UI. Тесты покрывают полный жизненный цикл покупки от регистрации до оплаты через Stripe.

4.2. Результаты тестов

Тестовая база проекта включает в себя **428 тестов**. При запуске команды `./mvnw test` все тесты выполняются успешно с итогом `'BUILD SUCCESS'`.

Таблица 15. Распределение тестов по классам

Тестовый класс	Количество тестов	Тип	Описание
`ModelExtendedTest`	161	Unit	Тестирование геттеров, сеттеров и конструкторов JPA-сущностей.
`ModelValidationTest`	37	Unit	Валидация ограничений полей (@NotEmpty, @Email, @Min, @Size).
`ServiceExtendedTest`	70	Integration	Тесты бизнес-логики сервисов с Mockito-заглушками.
`UserServiceImplTest`	14	Unit	Проверка регистрации, блокировки, разблокировки аккаунтов.
`CartServiceImplTest`	9	Unit	Логика работы корзины, пересчёт сумм, обновление количества.
`CategoryServiceImplTest`	10	Unit	Управление категориями товаров, деактивация.

`ProductOrderServiceImplTest`	10	Unit	Оформление заказов, шифрование транзакций, сохранение.
`ProductServiceImplTest`	11	Unit	Управление товарами, загрузка изображений, скидки.
`SecurityUtilsTest`	15	Unit	Тесты AES-шифрования и дешифрования ID платежей Stripe.
`CommonUtilsTest`	5	Unit	Тесты отправки email, генерации токенов сброса пароля.
`AppConstantTest`	1	Unit	Проверка констант системы (UNLOCK_DURATION_TIME).
`EcommerceE2eTest` (Playwright)	85	E2E	Полные сценарии покупки в реальном браузере Chromium.
ИТОГ	428 / 428	—	Все тесты пройдены — BUILD SUCCESS

4.2.1. Модульное и интеграционное тестирование

Модульные тесты изолированно проверяют поведение классов без зависимостей от внешних систем. Пример архитектуры теста `CartServiceImplTest`:

```
@ExtendWith(MockitoExtension.class)
class CartServiceImplTest {
    @Mock CartRepository cartRepository;
    @Mock ProductRepository productRepository;
    @Mock UserRepository userRepository;
    @InjectMocks CartServiceImpl cartService;

    @Test
    void testSaveCart_NewProduct_ShouldCreateCartWithQuantityOne() {
        // Arrange: настройка заглушек
        when(userRepository.findById(1L)).thenReturn(Optional.of(testUser));
        when(productRepository.findById(1L)).thenReturn(Optional.of(testProduct));
        when(cartRepository.findByProductIdAndUserId(1L, 1L)).thenReturn(null);
    }
}
```

```

        // Act: вызов тестируемого метода
        Cart result = cartService.saveCart(1L, 1L);

        // Assert: проверка результата
        assertEquals(1, result.getQuantity());
        verify(cartRepository, times(1)).save(any(Cart.class));
    }
}

```

Интеграционные тесты с H2 используют аннотацию `@SpringBootTest` с профилем `test`, который переключает источник данных на встроенную in-memory базу:

```

# application-test.properties
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop

```

Класс `ModelExtendedTest` содержит 161 тест на геттеры и сеттеры всех JPA-сущностей, что гарантирует корректность маппинга данных между объектами и таблицами базы данных. Класс `SecurityUtilsTest` содержит 15 тестов, проверяющих симметричность шифрования: зашифрованный ID должен корректно расшифровываться обратно, а шифрование одного и того же значения должно давать разные результаты (благодаря случайному вектору инициализации IV).

4.2.2. Автоматизированное E2E тестирование (Playwright)

Использование **Microsoft Playwright** вместо устаревшего Selenium позволило ускорить браузерные тесты в несколько раз благодаря встроенным механизмам ожидания (auto-waiting) и поддержке современных браузерных API.

Конфигурация Playwright:

- Браузер: Chromium (headless-режим для CI/CD).
- Точка входа: локальный сервер на `http://localhost:8080`.
- Запись видео: включена (`recordVideo: { dir: "test-results/videos" }`).

Архитектура E2E-теста `EcommerceE2eTest`:

Класс `EcommerceE2eTest` следует паттерну `@BeforeAll / @AfterAll` для управления жизненным циклом браузера:

- `@BeforeAll` — запуск Spring Boot приложения и браузера Playwright.
- Каждый тестовый метод — независимый сценарий.
- `@AfterAll` — закрытие браузера и остановка приложения.

Тест `testFullUserPurchaseLifecycle` имитирует полный жизненный цикл покупки:

1. Открытие главной страницы.
2. Регистрация нового тестового пользователя.
3. Поиск товара и добавление в корзину.
4. Переход на страницу оформления заказа.
5. Заполнение формы адреса и выбор терминала Omniva.
6. Заполнение тестовых данных карты Stripe (4242 4242 4242 4242).
7. Подтверждение платежа и верификация перенаправления на страницу успеха.
8. Проверка наличия заказа в истории покупок.

Скриншот кода сквозного E2E-теста:

```

1  @Test
2  @Order(1)
3  @DisplayName("E2E 1: Registration -> Login -> Select Product -> Cart -> Checkout (Stripe Card Payment) -> Order History")
4  void testFullUserPurchaseLifecycle() {
5      try {
6          String baseUrl = "http://localhost:" + port;
7
8          // — 1. Register a new user —
9          page.navigate(baseUrl + "/register");
10         page.waitForLoadState();
11
12         page.fill("input[name='name']", "E2E Tester One");
13         page.fill("input[name='email']", "e2etest1@example.com");
14         page.fill("input[name='mobile']", "+79991234567");
15         page.fill("input[id='password']", "SecurePassword@123");
16         page.fill("input[id='confirmPassword']", "SecurePassword@123");
17
18         // Submit registration form
19         page.click("button[type='submit']");
20
21         // Redirect to login page
22         page.waitForURL("**/signin*");
23         assertThat(page.url()).contains("/signin");
24
25         // — 2. Login —
26         page.waitForSelector("input[name='username']");
27         page.fill("input[name='username']", "e2etest1@example.com");
28         page.fill("input[name='password']", "SecurePassword@123");
29         page.click("button[type='submit']");
30
31         // After login, redirect to home page
32         page.waitForURL(baseUrl + "/"");
33         page.waitForSelector("span:has-text('E2E Tester One')");
34         assertThat(page.locator("span:has-text('E2E Tester One')").isVisible()).isTrue();
35
36         // — 3. Browse shop and select product —
37         page.navigate(baseUrl + "/products");
38         page.waitForLoadState();
39         page.waitForSelector("p.product-card-title");
40
41         // Check that at least one product from DataSeeder is displayed
42         Locator productTitle = page.locator("p.product-card-title").first();
43         assertThat(productTitle.textContent()).contains("Creatine 100% Pure");
44
45         // Go to first product details page
46         page.locator("p.product-card").first().click();
47         page.waitForURL("**/product/*");
48
49         // — 4. Add product to cart —
50         page.waitForSelector("button.add-to-cart-btn");
51         page.click("button.add-to-cart-btn");
52
53         // Verify product is added and cart badge updates to 1
54         page.waitForSelector("#cart-count:has-text('1')");

```

Рисунок. Реализация сквозного E2E теста на Playwright

В процессе выполнения тестов Playwright открывает реальный браузер Chromium и воспроизводит действия пользователя. На скриншоте ниже показан процесс выполнения сквозных тестов:

После завершения прохождения всех тестов в консоли Maven отображается успешный статус сборки проекта:

Покрытые E2E-сценарии (85 тестов):

Таблица 16. E2E тестовые сценарии Playwright

№	Группа сценариев	Количество тестов
1	Навигация и просмотр каталога (гость)	8
2	Поиск товаров	5
3	Регистрация пользователя (валидация форм)	12
4	Авторизация и выход из системы	7
5	Блокировка аккаунта при неверном пароле	4
6	Управление корзиной (добавление, изменение, удаление)	14
7	Оформление заказа и выбор терминала доставки	10
8	Полный цикл оплаты через Stripe	8
9	История заказов и личный кабинет	6
10	Административная панель (CRUD товары, заказы)	11
Итого		85

4.2.3. Встроенный сервис просмотра записей тестирования

Во время прохождения тестов Playwright записывает видеоролики выполнения каждого сценария в формате `.webm`. В приложении SPSHOP реализован уникальный встроенный интерфейс для разработчиков:

- По адресу `http://localhost:8080/test-videos` доступен дашборд просмотра тестов.

- На странице отображается список всех последних видеозаписей тестовых запусков.
- Каждое видео можно воспроизвести прямо в браузере без скачивания файла.
- Интерфейс обслуживается контроллером `TestVideoController`, который сканирует директорию `test-results/videos` и отдаёт список файлов.

Это значительно упрощает отладку UI-ошибок: разработчик может визуально проследить весь поток взаимодействия браузера с приложением на момент возникновения сбоя.

ЗАКЛЮЧЕНИЕ

В результате выполнения проекта была успешно спроектирована и реализована полнофункциональная веб-платформа электронной коммерции **SPSHOP**, специализирующаяся на розничной продаже спортивного питания на рынке Эстонии.

Применение современного технологического стека на базе **Java / Spring Boot 3.x** позволило создать высокопроизводительное, масштабируемое и устойчивое к нагрузкам решение корпоративного уровня. Многослойная MVC-архитектура с чётким разделением ответственности между слоями обеспечивает высокую сопровождаемость кода и простоту дальнейшего развития системы.

Интеграция с платежным шлюзом **Stripe** (через официальный Java SDK 24.0.0 и PaymentIntent API) обеспечила высочайший стандарт финансовой безопасности: данные платёжных карт клиентов не проходят через серверы приложения, а транзакционные идентификаторы хранятся в базе данных в зашифрованном виде. Это соответствует требованиям PCI DSS SAQ A.

Встроенная поддержка почтовых служб Эстонии — **Omniva** и **SmartPost** — с выбором конкретного терминала делает процесс доставки максимально прозрачным и удобным для конечного пользователя. Реализованный дизайн в стиле Glassmorphism Dark UI создаёт визуально привлекательный премиальный интерфейс.

Проект выделяется исключительным качеством кода и стабильностью, подтверждёнными полным покрытием **428 автоматизированными тестами** трёх уровней (модульные, интеграционные и E2E). Наличие встроенного видеосервиса Playwright-тестирования

является нестандартным техническим решением, значительно упрощающим процесс отладки интерфейса.

Разработанная система развёрнута в облаке Microsoft Azure и готова к промышленной эксплуатации. Потенциальными направлениями дальнейшего развития являются: добавление мультиязычности (i18n), программа лояльности, интеграция Apple Pay / Google Pay через Stripe, а также внедрение мобильного приложения на базе Spring REST API.

ИСПОЛЬЗУЕМЫЕ ИСТОЧНИКИ

1. Официальная документация Spring Framework и Spring Boot:
<https://spring.io/projects/spring-boot>
2. Руководство по Spring Security и авторизации: <https://spring.io/projects/spring-security>
3. Документация Hibernate ORM и Jakarta Persistence:
<https://hibernate.org/orm/documentation/>
4. Документация платежной системы Stripe — PaymentIntent API:
[https://stripe.com/docs/api/payment_intents](https://stripe.com/docs/api/payment_intents)
5. Документация Stripe Java SDK:
<https://stripe.com/docs/api?lang=java>
6. Руководство по Stripe Elements и PCI DSS:
<https://stripe.com/docs/security/guide>
7. Руководство разработчика по Microsoft Playwright Java:
<https://playwright.dev/java/docs/intro>
8. Справочник по разметке HTML5 и Thymeleaf:
<https://www.thymeleaf.org/documentation.html>
9. Инструкции по разворачиванию в Microsoft Azure App Service:
<https://learn.microsoft.com/en-us/azure/app-service/>
10. Документация Docker для контейнеризации Java-приложений:
<https://docs.docker.com/language/java/>
11. Официальная документация JUnit 5: <https://junit.org/junit5/docs/current/user-guide/>

12. Документация Mockito Framework: <https://site.mockito.org/>

13. Информация о почтовой службе Omniva Эстонии:

<https://www.omniva.ee/>

14. Информация о почтовой службе SmartPost:

<https://www.smartpost.ee/>